Analytics Vidhya

BLACKBELT DATA SCIENCE

CAPSTONE PROJECT

Drug Sentiment Analysis

NLP • Machine Learning • Text Classification

PROJECT OVERVIEW

Predict drug sentiment from patient comments

Total Marks: 100 • Submission: Code + Presentation + predictions.csv

1. Problem Statement

Sentiment analysis can extensively enhance the way we look at data and customer opinions, with the benefit of reduced manual effort. Sentiment Analysis is used in a wide variety of industries, and can also help pharmaceutical companies easily work with large volumes of unstructured data to gain useful insights regarding their products.

You are the lead Data Scientist for the pharmaceutical company Kipla. They have collected comments for various products scraped from various online sources and wish to create a sentiment analysis engine that can track the sentiment regarding a specified drug from 3 categories:

0 — POSITIVE	1 — NEGATIVE	2 — NEUTRAL

Key Challenge

Every sample will contain the text mentioning a drug name and the comment pertaining to it. Multiple products could be present within a single comment, and the sentiment must be predicted per drug-comment pair.

Important: Each row represents a specific (drug, text) pair. Your model must predict sentiment toward the drug named in that row, not the overall sentiment of the full comment. The same comment text may appear with different drugs and can have different labels depending on which drug is being evaluated.

Illustrative Example

> *"I looked up stomach pain after taking Correctol...and here I am, wishing I had read these comments before I took it. I'm sitting here with the worst stomach pain I've had since labor. I feel like I need to vomit but have dry mouth. My intestines feel like someone is twisting them. I've honestly been sitting on the toilet for an hour... I will NEVER take this medication again! Post that I took Meftal Spas and it worked wonders for me."*

From the above comment:

- Meftal Spas → Sentiment: POSITIVE (0)
- Correctol → Sentiment: NEGATIVE (1)

This illustrates that the task is drug-conditioned sentiment classification. A model that predicts one sentiment for the whole comment without considering the target drug is not solving the task correctly.

2. Data Description

You are provided with the following files:

2.1 train.csv (5,619 rows)

Contains labelled drug–comment pairs for model training.

Column	Type	Description
unique_hash	String (ID)	Unique identifier for each record
text	String	The full comment text mentioning the drug
drug	String	Drug/product name the sentiment is about
sentiment	Integer (Target)	0 = Positive, 1 = Negative, 2 = Neutral

2.2 test.csv (3,107 rows)

Contains drug–comment pairs without labels. Participants must predict the sentiment for each row.

Column	Type	Description
unique_hash	String (ID)	Unique identifier for each record
text	String	The full comment text mentioning the drug
drug	String	Drug/product name to classify sentiment for

2.3 sample_submission.csv

Shows the required format for the final prediction file. Only two columns are required:

Column	Description
id	The unique_hash from test.csv
sentiment	Predicted sentiment: 0, 1, or 2

⚠ *Important: The submission file must contain ONLY the 'id' and 'sentiment' columns. Any additional columns will invalidate your submission.*

In the submission file, the id column must contain the values from the unique_hash column of test.csv in the same row order as the test set, unless otherwise instructed.

3. Evaluation Metric

Model performance will be evaluated using the Weighted F1-Score across all three classes (Positive, Negative, Neutral).

Metric	Value / Formula
Primary Metric	Weighted F1-Score
Classes	0 (Positive), 1 (Negative), 2 (Neutral)
Higher is Better	Yes (Range: 0.0 – 1.0)

The weighted F1-score accounts for class imbalance by weighting the F1-score of each class by its support (number of true instances). This means a model that performs better on majority classes will naturally score higher — so addressing class imbalance is important.

Students should report both the overall Weighted F1-score and the class-wise precision, recall, and F1-score during validation. This is important because a high weighted score can still hide weak performance on minority classes.

4. Suggested Approach & Steps

The following steps outline a recommended pipeline. Students are encouraged to experiment beyond these suggestions.

Minimum expected project scope: at least 5 meaningful EDA visualisations, at least 3 models trained and compared, and a clear justification for the final selected model.

Step 1 — Data Loading & Exploration (EDA)

1. Load train.csv, test.csv, and sample_submission.csv using pandas.
2. Check dataset shapes, data types, and null values.
3. Analyse the class distribution of the target variable (sentiment).
4. Explore the distribution of drug names in both train and test sets.
5. Visualise comment lengths (characters and word count).
6. Identify common words per sentiment class using word clouds or frequency plots.

> 💡 *Key questions to answer during EDA: • Is the dataset class-imbalanced? • Are there any drugs only in test but not in train? • What is the average comment length? Does it differ by sentiment?*

Step 2 — Text Preprocessing

Choose preprocessing based on the model family you use. For TF-IDF and Bag-of-Words models, text cleaning and normalisation may help. For transformer-based models, use minimal preprocessing and preserve the original text as much as possible.

7. Normalise text only as needed for your chosen model.
8. For classical NLP models, you may remove HTML tags, URLs, and noisy characters.
9. Tokenise only if required by your method.
10. Remove stop words only if it improves validation performance. Be careful with negations such as “not”, “never”, and “no”.
11. Apply stemming or lemmatisation only if justified. Do not assume these will help all models.
12. Ensure the model is explicitly aware of the target drug for each row. This can be done in different valid ways, such as concatenating the drug with the text, using the drug as a separate feature, highlighting the target drug within the text, or designing a model input that conditions prediction on the drug-text pair.

Step 3 — Feature Engineering

Option A: Traditional NLP Features

- TF-IDF (Term Frequency-Inverse Document Frequency) on text
- Bag-of-Words (CountVectorizer)
- N-grams (bigrams, trigrams)
- Sentiment lexicon features (VADER, TextBlob scores)

- Comment length, word count, average word length
- Drug name as a categorical feature (one-hot or embedding)

Option B: Pre-trained Word Embeddings

- Word2Vec / GloVe / FastText averaged embeddings
- Drug-specific features: is the drug name mentioned more than once?

Option C: Transformer-based Features (Advanced)

- BERT / RoBERTa / DistilBERT sentence embeddings
- Fine-tune a pre-trained transformer directly on this task

Step 4 — Model Building

Start with simple baselines and progressively build more complex models.

Model Type	Examples	Recommended For
Baseline	Majority class, Random	Benchmark comparison
Classical ML	Logistic Regression, SVM, Naive Bayes, Random Forest, XGBoost	TF-IDF / BoW features
Deep Learning	LSTM, BiLSTM, CNN-LSTM	Embedding features
Transformers	BERT, DistilBERT, RoBERTa (fine-tuned)	Best performance

Step 5 — Handling Class Imbalance

- First inspect class distribution in the training set.
- Use stratified validation for reliable comparison.
- Try class-weighted models where appropriate.
- Oversampling or undersampling may be used only if applied correctly within the training pipeline and justified using validation results.
- Advanced techniques such as threshold tuning are optional and should only be used if students can clearly explain and validate them.

Step 6 — Model Evaluation & Selection

- Use stratified k-fold cross-validation (k=5 recommended).
- Report precision, recall, and F1-score per class.
- Use confusion matrix to understand misclassifications.
- Compare models using the Weighted F1-Score.

Use only train.csv for model development, validation, model selection, and hyperparameter tuning. Do not use test.csv for any decision-making. The test set is only for final prediction generation.

Step 7 — Prediction & Submission

13. Apply all preprocessing steps to the test set (fit only on train!).
14. Generate predictions using the best model.
15. Create the submission file with columns: id, sentiment.
16. Validate that predictions only contain values 0, 1, or 2.
17. Submit as a .csv file matching the sample_submission.csv format.

⚠ *Critical: Never fit any preprocessing, vectorization, scaling, encoding, feature selection, or resampling step using information from test.csv. During validation, all such steps must be fit only on the training fold and then applied to the validation fold. After final model selection, fit the pipeline on the full training data and apply it to test.csv.*

5. Deliverables

You are required to submit the following two items:

5.1 Code File (Jupyter Notebook / Python Script)

- A well-structured, reproducible Jupyter Notebook (.ipynb) or Python script (.py).
- The notebook should run end-to-end without errors (restart & run all).
- Clearly organised into sections matching the approach steps.
- Includes markdown cells explaining each step and key decisions.
- Produces the final predictions.csv file.

5.2 Presentation (PowerPoint / PDF)

A presentation (10–15 slides) covering:

The slide structure below is a recommended format, not a mandatory slide-by-slide template. Students may adjust the structure as long as all required sections are clearly covered.

Slide #	Section	Content
1	Title Slide	Project title, your name, date
2	Problem Understanding	Business problem, objective, and target variable
3–4	EDA & Data Insights	Key visualisations and findings from exploration
5–6	Data Preparation & Feature Engineering	Preprocessing steps, features created, rationale
7–8	Modelling Approach	Models tried, evaluation methodology, results table
9	Best Model & Performance	Final model, F1-score breakdown, confusion matrix
10	Challenges & Learnings	Key challenges faced and how they were resolved
11	Conclusion	Summary and potential improvements

5.3 Submission File

- predictions.csv (or as instructed) matching sample_submission.csv format.
- Columns: id (unique_hash), sentiment (0 / 1 / 2).
- Total rows must equal the number of rows in test.csv (3,107 rows).

6. Evaluation Rubrics (Total: 100 Marks)

6.1 Project Component (60 Marks)

Sub-Category	Marks	What Evaluators Look For
Problem Understanding & Data Preparation	15	Correct understanding of the task. Thorough EDA with visualisations. Proper handling of missing values, duplicates, and class imbalance. Validation strategy within train.csv with no data leakage. test.csv must not be used for model selection or tuning.
Feature Engineering	15	Creativity and relevance of features. Correct use of NLP techniques (TF-IDF, embeddings, etc.). Drug name as a signal. Justification for feature choices.
Model Building	15	Multiple models tried and compared. Correct evaluation using Weighted F1-Score. Hyperparameter tuning. Final model selection with clear rationale.
Code Quality	15	Clean, readable, and well-commented code. Logical structure and modularity. No redundant code. Reproducible results.

6.2 Technical Understanding (40 Marks)

Evaluated during viva / presentation Q&A:

Area	Marks	Topics Covered
Python	10	Python proficiency, use of pandas/numpy/sklearn/NLP libraries, efficient code.
Statistics & EDA	10	Understanding of class distribution, imbalance, feature patterns, text characteristics, and correct interpretation of visualisations and validation results.
Feature Engineering	10	Ability to explain feature choices, NLP pipeline decisions, handling of text data.
ML, DL & NLP	10	Understanding of model internals, evaluation metrics, NLP concepts (TF-IDF, embeddings, transformers).

Component	Marks
Project (Problem Understanding + Feature Engineering + Model Building + Code Quality)	**60**
Technical Understanding (Python + Statistics & EDA + Feature Engineering + ML/DL/NLP)	**40**

TOTAL	100

7. Tips, Best Practices & Common Mistakes

Best Practices

- Always start with a simple baseline model before building complex ones.
- Use stratified splits and cross-validation to get reliable F1-score estimates.
- Fit all preprocessing transformers (TF-IDF, encoders, scalers) only on training data.
- Document every decision in your notebook with markdown explanations.
- Save your trained models (use joblib or pickle) so you don't need to retrain.
- Version your experiments — keep a table of model name, features, and F1-score.
- Check that the test set predictions have exactly 3,107 rows before submission.

Common Mistakes to Avoid

- Data leakage: fitting transformers on the full dataset (train + test combined).
- Ignoring class imbalance — always check and address it.
- Evaluating on the training set only — always validate on a held-out set.
- Submitting predictions with wrong column names or extra columns.
- Failing to make the model aware of the target drug for each row.
- Over-engineering features early — start simple, iterate.

Recommended Libraries

Purpose	Libraries
Data Manipulation	pandas, numpy
Visualisation	matplotlib, seaborn, wordcloud, plotly
Text Preprocessing	nltk, spacy, re (regex)
Classical ML Features	scikit-learn (TfidfVectorizer, CountVectorizer)
ML Models	scikit-learn, xgboost, lightgbm
Deep Learning / NLP	tensorflow/keras, pytorch, transformers (HuggingFace)
Sentiment Lexicons	vaderSentiment, textblob
Class Imbalance	imbalanced-learn (SMOTE)
Evaluation	scikit-learn (classification_report, f1_score)

8. Submission Checklist

Use this checklist before submitting your capstone to ensure you have covered all required components.

Code File

- Notebook runs end-to-end without errors
- All sections clearly labelled
- EDA section with at least 5 meaningful visualisations
- Data preprocessing pipeline clearly documented
- At least 3 different models trained and compared
- Cross-validation used for model evaluation
- Final model selected with clear justification
- Clear explanation of how the target drug was incorporated into the modeling pipeline
- Predictions generated for all 3,107 test rows

Presentation

- 10–15 slides covering all required sections (see Section 5.2)
- Visualisations included in slides
- Results table comparing models
- Final model performance and confusion matrix

Submission File

- File has exactly 2 columns: id and sentiment
- Total rows = 3,107 (matching test.csv)
- Sentiment values are only 0, 1, or 2 (no floats, no NaN)
- unique_hash IDs match those in test.csv exactly

Good luck!

This capstone is designed to test your end-to-end Data Science and NLP skills. Focus on understanding the problem deeply, building a clean and explainable pipeline, and iterating towards a strong model. Quality of thinking and process matters as much as the final score.